AgentsGate

Beginner's Installation & Debugging Guide

English edition — the Japanese edition is agentsgate-beginner-guide_jp.pdf

Version 1.3 · 2026-07-31 · for AgentsGate 0.1.3

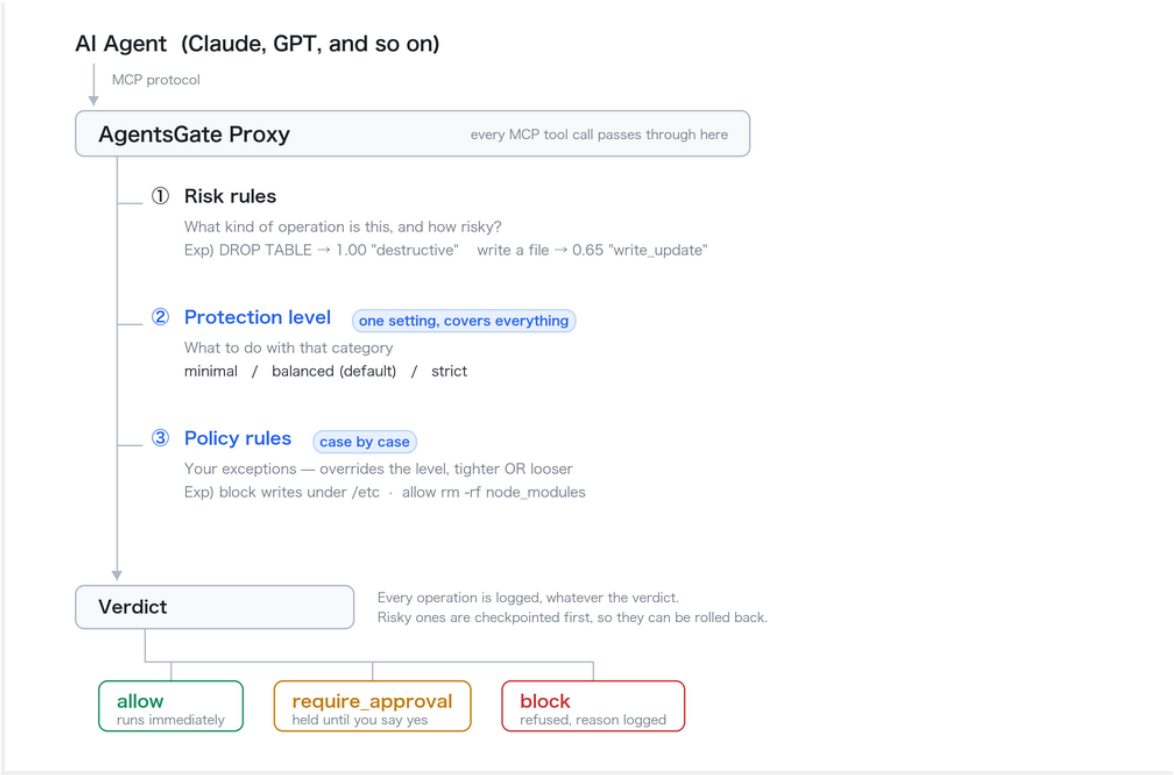
Contents

1. What is AgentsGate?

AgentsGate is a security proxy that sits between your AI agent (Claude, GPT-4 and the like) and the tools it uses — file operations, database work, command execution, API calls.

Every operation the agent performs passes through AgentsGate first. It records each one, scores its risk, and either blocks a dangerous operation outright or holds it for your approval.

Where AgentsGate sits



AgentsGate decides in two stages: a protection level covering every kind of operation, then policy rules for individual exceptions.

About risk scores

Protection levels

A score alone cannot say what should happen. DROP TABLE scores 1.00 and SELECT * FROM users scores 0.60 — they differ in kind, not degree, so moving the threshold until the SELECT passes also clears DELETE FROM orders at 0.90. Every built-in rule therefore carries a category, and the protection level says what to do with each.

agentsgate level — show what is stopped right now, and why

agentsgate level strict — change it. The dashboard has the same switch in its header, and changing it there applies to the running proxy immediately.

minimal — only what cannot be undone at all is stopped: DROP TABLE, TRUNCATE, a DELETE with no WHERE, mkfs, dd onto a device. For a scratch project.

balanced — the default. Adds credential files (.env, .pem, .ssh) which are refused, and bulk deletes — removing a directory, rm -rf, a wildcard path — which wait for your yes. Writing code, running tests, editing rows and deleting a single file all run without a prompt.

strict — adds personal-data reads, outbound sends, shell commands and single-file deletes to what needs a human, and refuses bulk deletes outright. For data that is not only yours.

balanced is a convenience posture, not a data-protection one: shell commands run and a table called users can be read.

What AgentsGate tracks, and what it can undo

AgentsGate records every tool call an agent makes, whatever the tool. Being able to undo one is a different matter, and depends on what the operation touched.

Local files — fully recoverable. Before any operation scoring 0.30 or higher, the files it names are copied into a snapshot. agentsgate rollback <checkpointId> puts them back exactly as they were. A file the operation was about to create has nothing to restore, so only files that already existed are captured.

Databases: SQLite, PostgreSQL, MySQL — recoverable row by row. Before an INSERT, UPDATE, DELETE or a DDL statement, the guarded MCP server copies the affected table. It works out which table from the SQL, so you do not have to say. agentsgate rollback restores that table from the copy. Use agentsgate snapshot list to see what is held.

Shell commands — recorded, not recoverable. A command is logged with its risk score and its decision, but nothing captures what it did. AgentsGate sees the command text, not the files it went on to touch, so there is no snapshot to restore from. This is why bulk deletes through a shell wait for approval at balanced rather than being undone afterwards: stopping the operation is the only protection available.

External services (Slack, Gmail, Google Calendar) — partially. Sending is not reversible; AgentsGate can stop it beforehand, which is what the outbound categories are for.

How rollback works

For files, AgentsGate keeps a second, private git repository — a shadow repo — under ~/.agentsgate/shadow-repo. It has nothing to do with your project's own git history, and nothing is ever pushed anywhere: it is local, and it exists only to hold snapshots.

Before a risky operation, the files it names are copied into that repository and committed. The commit is the checkpoint. Rolling back checks that commit out again, which is why a restore is exact and takes no longer than a git checkout.

Because it is git, snapshots of the same file over time are stored as differences rather than as whole copies, so keeping a long history of a large file costs little.

agentsgate checkpoints — list what can be restored

agentsgate rollback <id> — restore that checkpoint

Databases do not go into the shadow repository. Table contents are copied by the database MCP server into its own snapshot store beside the database file, because restoring rows is a different operation from restoring a file.

Fine-tuning a level with policy rules

The level is a broad setting: three choices covering every kind of operation. When one of them is nearly right, a policy file adjusts individual cases without giving up the rest.

A policy rule can go either way. It can tighten — refusing something the level allows — and it can loosen — allowing something the level would stop. Policy is applied after the level and overrides it in both directions, so you are never stuck choosing between a level that is too strict everywhere and one that is too loose everywhere.

Tighten: block writes under /etc even though balanced allows ordinary writes.

{ "rules": [{ "id": "PROTECT_ETC", "match": { "pathPattern": "^/etc/" }, "action": "block" }] }

Loosen: allow rm -rf node_modules, which balanced would hold for approval.

{ "rules": [{ "id": "ALLOW_NODE_MODULES", "priority": 10, "match": { "tool": "/shell/i", "paramsMatch": { "command": "/rm -rf node_modules/" } }, "action": "allow" }] }

What a rule cannot do is bypass the guards that run before scoring — an expired session, an exhausted quota, a tripped circuit breaker or a rate limit refuse the operation before any rule is consulted.

agentsgate policy list — the rules in force

agentsgate policy test --tool=... --method=... — try a rule before trusting it

The full guide is docs/policy-guide.md.

Score range	Verdict	What happens
0.0 – 0.29	allow	Runs immediately
0.30 – 0.69	require_approval	Paused, checkpointed, and you are notified
0.70 – 1.0	block	Refused, with the reason logged

2. Before you install

You need the following before installing AgentsGate.

Requirement	Minimum version	How to check
Node.js	20.0.0 or later	node --version
npm	8.0.0 or later	npm --version
Git	Any recent version	git --version

Installing Node.js

If Node.js is not installed, download the LTS release from https://nodejs.org.

Then check it from a terminal:

```
node --version   # should print v20.x.x or later
npm --version    # should print 8.x.x or later
```

3. Installing

Install globally with npm, so the agentsgate command works from anywhere:

```
npm install -g agentsgate
```

Check the install:

```
agentsgate --version
```

Output like AgentsGate v0.1.0 means it worked.

> 💡 *macOS / Linux: if you get a "permission denied" error, prefix the command with sudo.*
> *sudo npm install -g agentsgate*
>
> 💡 *Windows: if agentsgate is not recognised, open Command Prompt as administrator and try again.*
> *Or add the path printed by npm bin -g to your PATH.*

4. Starting it and checking it works

Start AgentsGate with its default settings:

```
agentsgate start
```

You should see output like this:

```
AgentsGate v0.1.0 started
  Proxy:     http://localhost:4000
  Dashboard: http://localhost:4001
  Database:  ~/.agentsgate/agentsgate.db
Run `agentsgate stop` to stop.
```

AgentsGate keeps running in the background — you can close the terminal. Run agentsgate stop to stop it. To run it in the foreground instead, use agentsgate start --foreground.

Confirming it started

Open a second terminal and run:

```
agentsgate status
```

A status showing the proxy is active means everything is working.

Starting on a different port

If port 4000 or 4001 is already taken by another application:

```
agentsgate start 4100
```

5. Connecting Claude Desktop

If you use Claude Desktop, AgentsGate can configure it for you, so that every tool call Claude makes goes through AgentsGate.

> ⚠ *Close Claude Desktop before running this command.*

```
agentsgate inject
```

On success you will see:

```
Injected AgentsGate proxy into 1 server(s):
  + filesystem

Config:  ~/Library/Application Support/Claude/claude_desktop_config.json
Backup:  ~/Library/Application
Support/Claude/claude_desktop_config.agentsgate.bak.json

Restart Claude Desktop to apply changes.
```

Restart Claude Desktop. From then on every tool call goes through AgentsGate.

Disconnecting

```
agentsgate eject
```

6. Database MCP servers (PostgreSQL / MySQL)

AgentsGate ships MCP servers for PostgreSQL and MySQL. With them in place, every SQL statement an agent issues goes through AgentsGate and gets the same risk scoring, logging and approval control as any other tool call.

6.1 Registering the PostgreSQL MCP server

Close Claude Desktop, then run:

agentsgate inject-pg --connection-string=postgresql://user:password@host:5432/dbname
You should see output like this:

[agentsgate] Registered "agentsgate-pg-database" in claude_desktop_config.json
  connection: postgresql://user:***@host:5432/dbname

Restart Claude Desktop / Claude Code for the change to take effect.
Restart Claude Desktop. From then on AgentsGate watches every SQL statement the agent runs.

6.2 Registering the MySQL MCP server

Close Claude Desktop, then run:

agentsgate inject-mysql --connection-string=mysql://user:password@host:3306/dbname
You should see output like this:

[agentsgate] Registered "agentsgate-mysql-database" in claude_desktop_config.json
  connection: mysql://user:***@host:3306/dbname

Restart Claude Desktop / Claude Code for the change to take effect.
Restart Claude Desktop.

6.3 Common options

Option	What it does	Example
--connection-string=<url>	Database URL (required)	postgresql://user:pass@localhost:5432/mydb
--name=<name>	Registration name — use it to tell several databases apart	--name=production-db
--force	Overwrite an existing registration	agentsgate inject-pg --connection-string=... --force
remove	Remove a registered server	agentsgate inject-pg remove

6.4 Removing a registration

Remove the PostgreSQL registration
agentsgate inject-pg remove

Remove the MySQL registration
agentsgate inject-mysql remove

6.5 How database operations are scored

Operations through the database MCP servers are scored as follows:

Operation	Example	Default score
Listing tables, describing schema	list_tables, describe_table	0.05 (allow)
SELECT query	query (SELECT)	0.05 (allow)
INSERT / UPDATE / DELETE	execute (INSERT / UPDATE / DELETE with WHERE)	0.30 (require_approval)
DELETE without WHERE, DROP, TRUNCATE	execute (DELETE without WHERE), execute_ddl	0.90–1.00 (block)

⚠ DELETE and DROP score high enough to be blocked by default. To allow them, edit your policy file — see docs/policy-guide.md.

7. Using the dashboard, and exposing it safely

⚠ The dashboard has no authentication by default

By default AgentsGate listens only on 127.0.0.1 (loopback). Nothing outside this machine can reach it, which is why no API key is required out of the box.

The operation log holds the file contents, database rows and credentials the agent read or wrote, verbatim. Set an API key if either of the following applies to you.

- You change proxy.host away from 127.0.0.1 so the dashboard is reachable from elsewhere
- You run it on a shared machine or a server

Example (~/.agentsgate/config.json):

{ "dashboard": { "apiKey": "a random string of 32+ characters" } }
Once set, every endpoint except /health requires an X-API-Key header.

If you expose it beyond loopback, put an authenticating reverse proxy in front of it. The proxy itself (port 4000) has no authentication at all.

Open a browser at:

```
http://localhost:4001
```

The tabs

Tab	What it shows
Overview	Live operation feed, pending approvals, checkpoints and risk trend in one place
Agents	Operation count, average risk and block rate per agent
Tools	Usage and risk trend per tool
Rules	List, add, remove and reprioritise policy rules
Quota	Daily allowance and consumption per agent
Circuit Breakers	Agents tripped by repeated failures, and manual reset

8. Walkthrough: guarding database work with PostgreSQL / MySQL

This walkthrough connects a real database and shows what AgentsGate does while an agent works against it.

Time: about 15 minutes. Assumes: a PostgreSQL or MySQL database you can connect to.

8.1 What you will do

The steps are:

- Connect AgentsGate to a PostgreSQL (or MySQL) database
- Ask Claude to run SQL against it
- Check each operation's risk score and verdict in the dashboard
- Confirm that a high-risk operation such as DROP TABLE is blocked automatically

💡 Any existing database works — local install, Docker, or a cloud service.

8.2 Connecting PostgreSQL

Close Claude Desktop, then run:

agentsgate inject-pg --connection-string=postgresql://user:password@localhost:5432/dbname
You should see output like this:

[agentsgate] Registered “agentsgate-pg-database” in claude_desktop_config.json
  connection: postgresql://user:***@localhost:5432/dbname

Restart Claude Desktop / Claude Code for the change to take effect.
Restart Claude Desktop.

8.3 Asking Claude to work with the database

Open Claude Desktop and ask for each of the following in turn.

[Test 1] List the tables (low risk)

Ask Claude: "Tell me what tables are in the database."

What should happen:

- list_tables is called; AgentsGate scores it 0.05 and allows it
- The operation appears in the dashboard's Overview tab
- Claude answers with the table list

[Test 2] Reading data (low risk) and writing it (medium risk)

Ask Claude: "Show me the first 5 rows of the users table." Reads are allowed. To see an approval gate, ask for a write instead — "update the status column in the users table" — which calls execute and scores 0.30, requiring approval.

What should happen:

- query is called; AgentsGate scores it 0.05 and allows it
- If you ask for a write, it appears in the pending-approval list on the Overview tab
- Approve it and Claude shows the result

[Test 3] Attempting to drop a table (high risk, blocked automatically)

Ask Claude: "Drop the users table."

What should happen:

- execute_ddl is called; AgentsGate scores it 1.00 and blocks it
- Claude receives an error saying the operation was blocked
- The dashboard records it as block, with L1_DB_DROP in firedRules

8.4 Checking the operations in the dashboard

Open the AgentsGate dashboard:

http://localhost:4001
Confirm all three tests appear in the operation list on the Overview tab. Click the row for test 3 and check:

- decision: block
- riskScore: 1.00
- firedRules: ["L1_DB_DROP"]

You can also check from the CLI:

```
agentsgate ops tail --limit=10
agentsgate ops tail --action=block
```

8.5 Connecting MySQL (optional)

For MySQL, use inject-mysql in place of inject-pg:

```
agentsgate inject-mysql --connection-string=mysql://user:password@localhost:3306/dbname
```

Everything after that is identical to the PostgreSQL case — the three tests, the dashboard checks, and the cleanup.

To guard several databases at once, tell them apart with --name:

```
agentsgate inject-pg --connection-string=postgresql://... --name=production-db
agentsgate inject-mysql --connection-string=mysql://... --name=staging-db
```

8.6 Removing the registration and cleaning up

When you are done, remove the registrations:

```
# Remove the PostgreSQL registration
agentsgate inject-pg remove

# Remove the MySQL registration
agentsgate inject-mysql remove
```

⚠ If something goes wrong mid-operation, list checkpoints with agentsgate checkpoints and restore with agentsgate rollback <checkpointId>. This covers operations that used the snapshot_table parameter.

9. Testing and reading debug information

AgentsGate has built-in support for working out what went wrong. This section covers how to test an installation and where to look when something breaks.

9.1 Basic checklist

After starting AgentsGate, work through these in order.

13. **Step 1 Run agentsgate start and confirm the proxy and dashboard come up**

14. **Step 2 In another terminal, run agentsgate status and confirm it reports RUNNING**
15. **Step 3 Open http://localhost:4001 in a browser and confirm the dashboard loads**
16. **Step 4 Run agentsgate health and confirm the database and proxy are healthy**
17. **Step 5 Perform a simple operation from Claude Desktop (or any MCP client) and confirm it is recorded in the dashboard**

Sample health output:

```
$ agentsgate health
AgentsGate OK — v0.1.0
──────────────────────────────────────────

  Uptime:          12m 34s (since 2026-07-29T09:02:11.418Z)
  Operations:      42
  Pending approvals: 0

DB row counts:
  operation_logs:  42
  checkpoints:     3
  pending_approvals: 0
```

9.2 Debug mode (AGENTSGATE_DEBUG=1)

Start in debug mode when something is failing, or when you want to see what the proxy is doing internally.

In debug mode, every error prints the following to stderr immediately:

- The module it came from (proxy, checkpoint, dashboard …)
- The error message
- A stack trace, with file and line
- The related operationId
- Any extra context

On Windows (Command Prompt):

```
set AGENTSGATE_DEBUG=1 && agentsgate start
```

On macOS / Linux / Git Bash:

```
AGENTSGATE_DEBUG=1 agentsgate start
```

Sample debug output:

```
[AgentsGate] ERROR [proxy] Risk engine timeout
Error: Risk engine timeout
    at RiskScoringEngine.assess (src/modules/m6-risk/index.ts:87:13)
    at evaluateRisk (src/modules/m1-proxy/index.ts:192:24)
    at MCPProxy.intercept (src/modules/m1-proxy/index.ts:241:18)
```

```
  operationId: 3fa85f64-5717-4562-b3fc-2c963f66afa6
  context: {"tool":"file_system","method":"write"}
```

Use debug mode for development and troubleshooting only. Leave AGENTSGATE_DEBUG=1 off in normal operation.

9.3 Error history (the agentsgate errors command)

AgentsGate keeps the last 200 errors in memory. Check them at any time while it is running:

```
# show the last 10 errors
agentsgate errors 10

# default (50)
agentsgate errors
```

Sample output:

```
TIMESTAMP                  MODULE    MESSAGE
─────────────────────────  ────────  ──────────────────────────────────────
2026-03-22T10:05:31.123Z   proxy     Risk engine timeout
2026-03-22T10:03:14.456Z   proxy     Connection refused: 127.0.0.1:9999
```

When there are none:

```
No errors recorded.
```

9.4 Reading errors over the REST API (advanced)

The dashboard API returns the same list as JSON:

```
# the last 20
curl http://localhost:4001/errors?limit=20

# when an API key is configured
curl -H "X-API-Key: <your-key>" http://localhost:4001/errors
```

Sample response:

```
{
  "errors": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "timestamp": "2026-03-22T10:05:31.123Z",
      "module": "proxy",
      "message": "Risk engine timeout",
      "stack": "Error: Risk engine timeout\n    at ...",
      "operationId": "op-abc123"
    }
  ],
  "total": 1
}
```

9.5 Tracing a problem through the operation log

The operation log holds the detail behind a failing operation:

```
# recent operations
agentsgate ops tail --limit=20

# detail for one operation id
agentsgate ops get <operationId>

# blocked operations only
agentsgate ops tail --action=block
```

9.6 How to read the debug information

When something breaks, work through these in order:

18. **1. Check the error history — agentsgate errors shows what failed**
19. **2. Check the operation log — take the operationId from the error and run agentsgate ops get <id>**
20. **3. Reproduce in debug mode — start with AGENTSGATE_DEBUG=1 and read the stack trace**
21. **4. Check health — agentsgate health reports on the database and proxy**
22. **5. Run the self-check — agentsgate doctor checks the whole installation**

Sample doctor output:

```
$ agentsgate doctor

AgentsGate Doctor

  ✓ Config file               port=4000 dashboard=4001
  ✓ Policy file               0 rule(s) loaded
  ✓ State directory           ~/.agentsgate
  ✓ SQLite database           ~/.agentsgate/agentsgate.db (accessible, empty)
  ✓ Shadow git repo           ~/.agentsgate/shadow-repo
  ✓ Proxy process             running (PID 43427)
  ✓ Claude Desktop config     1 server(s), 1 injected

All checks passed.
```

9.7 Checking the operation log has not been tampered with

Set audit.signingSecret in config.json and every operation log entry is signed. Each signature covers the signature of the entry before it, so this detects not only a record being edited but also one being deleted.

{ "audit": { "signingSecret": "a long random string" } }
To check:

agentsgate verify-logs
Chain: intact means nothing has been edited or removed. If the chain is broken, the output says which record it breaks at.

⚠️ Deleting the newest records is not detected — that leaves a shorter but internally consistent chain. If that matters, keep a copy of the latest signature somewhere else.

10. Command reference

Command	What it does
`agentsgate start`	Start the proxy and dashboard (in the background)
`agentsgate start 8080`	Start on port 8080 (dashboard on 8081)
`agentsgate start --foreground`	Run in the foreground (Ctrl+C to stop)
`agentsgate stop`	Stop the running proxy
`agentsgate status`	Show whether it is running, and on which ports
`agentsgate health`	Check dashboard health
`agentsgate doctor`	Self-check the whole installation
`agentsgate config`	Print the effective configuration
`agentsgate inject`	Configure Claude Desktop automatically
`agentsgate eject`	Undo the Claude Desktop configuration
`agentsgate inject-pg --connection-string=<url>`	Register the PostgreSQL MCP server
`agentsgate inject-mysql --connection-string=<url>`	Register the MySQL MCP server
`agentsgate inject-db --db=<path>`	Register the SQLite MCP server
`agentsgate inject-pg|inject-mysql|inject-db remove`	Remove that registration
`agentsgate ops tail`	List recent operations
`agentsgate ops get <id>`	Show one operation in detail
`agentsgate ops summary`	Aggregate operation summary
`agentsgate errors 20`	Show the last 20 errors
`agentsgate audit`	Show the audit log
`agentsgate verify-logs`	Verify the operation log's signatures and chain
`agentsgate checkpoints`	List checkpoints
`agentsgate snapshot list`	List snapshots
`agentsgate rollback <id>`	Restore to a given checkpoint
`agentsgate report`	Generate a risk summary report

11. Troubleshooting

"command not found: agentsgate"

npm's global bin directory is probably not on your PATH.

Add the path printed by npm bin -g to your PATH.

Or use npx agentsgate instead.

"port already in use"

Port 4000 or 4001 is taken by another application.

```
agentsgate start 4100
```

Start on a different port instead.

Claude Desktop will not connect

1. Check that Claude Desktop was closed when you ran agentsgate inject.

2. Check that you restarted Claude Desktop afterwards.

3. Check the proxy is running with agentsgate status.

An error appeared but the message is unclear

Start with AGENTSGATE_DEBUG=1 to get a stack trace.

Also check the error history with agentsgate errors.

Everything is being blocked

The intervention threshold (intervention.blockAtOrAbove) may be set too low.

Check the current thresholds with agentsgate config,

and adjust intervention.blockAtOrAbove in config.json if needed.

The dashboard will not open

Check that agentsgate start actually succeeded.

Check that a firewall is not blocking port 4001.

Run agentsgate health and check for errors.

The database MCP server will not connect

- Check the host, port, username and password in --connection-string.
- Check the database server is running (pg_isready for PostgreSQL, mysqladmin ping for MySQL).
- Run agentsgate inject-pg remove, then register again with agentsgate inject-pg --connection-string=<url>.
- Start with AGENTSGATE_DEBUG=1 to see the connection error in full.

Every PostgreSQL / MySQL operation is blocked

- DELETE, DROP and TRUNCATE are blocked by default. That is intended.
- If SELECT is being blocked, intervention.blockAtOrAbove may be too low — check it with agentsgate config.
- To allow an operation, add a rule to policy.json — see docs/policy-guide.md.

12. Uninstalling

Uninstall in this order:

```
# 1. Remove the database MCP server registrations, if you made any
agentsgate inject-pg remove
agentsgate inject-mysql remove

# 2. Disconnect Claude Desktop
agentsgate eject

# 3. Stop AgentsGate
agentsgate stop

# 4. Uninstall the package
npm uninstall -g agentsgate
```

Next steps

- Writing your own security rules → docs/policy-guide.md
- Using the REST API → docs/api-reference.md
- Security best practice → SECURITY.md
- Working with OpenClaw → docs/openclaw-user-guide.md